

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
from os import path
path="/content/drive/MyDrive/MachineLearningPraktikum/Praktikum10/data"
```

```
df = pd.read_excel(path + "/Praktikum1.xlsx")
df
```

[Show hidden output](#)

```
df_temp = pd.read_excel(path + "/Praktikum1.xlsx", sheet_name="Sheet1", header=0)
df_train = df_temp.iloc[:, 5:9].head(8).copy()
df_train.columns = ['No', 'Suhu', 'Angin', 'Kelas']
```

```
# Konversi kolom ke tipe numerik
df_train['No'] = pd.to_numeric(df_train['No'])
df_train['Suhu'] = pd.to_numeric(df_train['Suhu'])
df_train['Angin'] = pd.to_numeric(df_train['Angin'])
```

```
# Data Uji untuk Marry
test_suhu = 16
test_angin = 3
```

```
from collections import Counter

# --- Definisi Fungsi KNN dan LOOCV ---

def knn_predict(X_train, y_train, X_test, k):
    """Memprediksi kelas untuk satu data uji menggunakan KNN."""
    # Hitung Jarak Euclidean
    distances = np.sqrt(np.sum((X_train - X_test)**2, axis=1))

    # Gabungkan jarak dan kelas
    df_temp = pd.DataFrame({'Jarak': distances, 'Kelas': y_train})
    df_temp_sorted = df_temp.sort_values(by='Jarak')

    # Ambil K tetangga terdekat dan tentukan kelas mayoritas
    top_k = df_temp_sorted.head(k)
    class_counts = Counter(top_k['Kelas'])
    return class_counts.most_common(1)[0][0]

def loocv_accuracy(df_data, k):
    """Menghitung akurasi dengan Leave-One-Out Cross-Validation."""
    X = df_data[['Suhu', 'Angin']].values
    y = df_data['Kelas'].values
    n = len(df_data)
    correct_predictions = 0

    for i in range(n):
        # Latih model dengan N-1 data, uji dengan data ke-i
        X_train = np.delete(X, i, axis=0)
        y_train = np.delete(y, i)
        X_test = X[i]
        y_test = y[i]

        predicted_class = knn_predict(X_train, y_train, X_test, k)

        if predicted_class == y_test:
            correct_predictions += 1

    return correct_predictions / n
```

```
# --- Menentukan K Optimal ---
```

```
k_values = range(1, len(df_train)) # K=1 sampai K=7
```

```
accuracy_scores = {}

for k in k_values:
    accuracy = loocv_accuracy(df_train, k)
    accuracy_scores[k] = accuracy

optimal_k = max(accuracy_scores, key=accuracy_scores.get)
```

```
# --- Klasifikasi Marry Menggunakan K Optimal ---

X_test_marry = np.array([test_suhu, test_angin])
X_train_data = df_train[['Suhu', 'Angin']].values
y_train_data = df_train['Kelas'].values

marry_prediction_optimal_k = knn_predict(X_train_data, y_train_data, X_test_marry, optimal_k)

print(f"Persepsi Marry (menggunakan K optimal={optimal_k}): {marry_prediction_optimal_k}")
```

Persepsi Marry (menggunakan K optimal=1): Dingin

Start coding or [generate](#) with AI.